

Pet Territory Wars — Mobile Architecture v1.0

Platform: Flutter — iOS + Android

Backend: Go REST API

Harita: Mapbox veya seçilecek uyumlu harita sağlayıcısı

Konum sistemi: Native location servisleri üzerinden Flutter adapter

Walk sonucu: Polling

Kapsam: Faz 1 Barebone Mobile MVP

Durum: Active

1. Amaç

Bu doküman Pet Territory Wars MVP mobil uygulamasının mimarisini tanımlar.

Mobil uygulamanın temel sorumlulukları:

- Kullanıcı oturumunu yönetmek
- Oyuncu ve pet bilgilerini göstermek
- Konum izinlerini yönetmek
- Walk başlatmak ve durdurmak
- GPS noktalarını güvenli şekilde toplamak
- Noktaları cihazda geçici olarak saklamak
- GPS noktalarını backend'e batch olarak göndermek
- Ağ kesintilerinde veri kaybını önlemek
- Walk işleme durumunu polling ile takip etmek
- Walk sonucunu kullanıcıya göstermek
- Territory hücrelerini haritada göstermek
- Leaderboard ve profil istatistiklerini sunmak

Mobil uygulama oyun sonucunu belirlemez.

```
Flutter
  ↓
GPS toplar
  ↓
Veriyi saklar ve gönderir
  ↓
Backend sonucu hesaplar
  ↓
Flutter sonucu gösterir
```

Authoritative kararlar her zaman Go backend ve Game Engine'e aittir.

2. Temel Mimari Karar

MVP mobil uygulaması feature-first ve katmanlı bir yapı kullanacaktır.

```
Presentation
  ↓
Application
  ↓
Data
  ↓
Platform / External Services
```

Her feature kendi içinde bu sorumlulukları ayırır.

Önerilen yapı:

```
lib/
├─ app/
├─ core/
├─ features/
└─ main.dart
```

Feature'lar:

```
auth
player
pets
cities
walk
territory
leaderboard
```

Flutter'ın resmi mimari yaklaşımı da UI ve data katmanlarının ayrılmasını, repository'lerin veri erişimini soyutlamasını ve UI durumunun ayrı yapılarda yönetilmesini tavsiye eder.

3. Proje Yapısı

```
lib/
├─ app/
│   ├─ app.dart
│   ├─ bootstrap.dart
│   ├─ router.dart
│   └─ dependencies.dart
```

```

└─ environment.dart

└─ core/
  └─ api/
    └─ api_client.dart
    └─ api_error.dart
    └─ api_response.dart
    └─ auth_interceptor.dart

  └─ auth/
    └─ token_storage.dart
    └─ session_manager.dart

  └─ database/
    └─ local_database.dart
    └─ migrations/

  └─ location/
    └─ location_service.dart
    └─ location_permission_service.dart
    └─ location_point.dart
    └─ location_settings.dart

  └─ network/
    └─ connectivity_service.dart
    └─ retry_policy.dart

  └─ polling/
    └─ polling_controller.dart

  └─ secure_storage/
  └─ logging/
  └─ config/
  └─ errors/

└─ features/
  └─ auth/
  └─ player/
  └─ pets/
  └─ cities/
  └─ walk/
  └─ territory/
  └─ leaderboard/

```

Bir feature'ın örnek yapısı:

```

features/walk/
└─ data/
  └─ walk_api.dart

```

```
|   |   |─ walk_local_source.dart
|   |   |─ walk_repository.dart
|   |   └─ dto/
|
|   |─ application/
|   |   |─ walk_controller.dart
|   |   |─ walk_state.dart
|   |   |─ start_walk.dart
|   |   |─ stop_walk.dart
|   |   └─ sync_walk.dart
|
|   |─ model/
|   |   |─ walk.dart
|   |   |─ walk_status.dart
|   |   |─ walk_result.dart
|   |   └─ pending_point_batch.dart
|
|   └─ presentation/
|       |─ walk_screen.dart
|       |─ walk_result_screen.dart
|       └─ widgets/
```

4. Katman Sorumlulukları

4.1 Presentation

Sorumlulukları:

- Widget oluşturmak
- Kullanıcı etkileşimlerini almak
- Application state'i göstermek
- Loading, error ve empty state göstermek
- Navigation başlatmak

Presentation katmanı:

- API çağrısı yapmaz
- Local database'e doğrudan erişmez
- GPS servisini doğrudan yönetmez
- Business karar üretmez

4.2 Application

Sorumlulukları:

- Ekran state'ini yönetmek

- Kullanıcı aksiyonlarını use case akışına çevirmek
- Repository çağırmak
- Walk lifecycle'ini koordine etmek
- Polling başlatmak ve durdurmak
- UI'ya gösterilecek durumu üretmek

Örnek:

```

Start Walk butonuna basıldı
  ↓
Permission kontrolü
  ↓
Backend Walk oluşturma
  ↓
Local Walk oluşturma
  ↓
GPS toplama başlatma
  ↓
UI state → WALKING

```

Application katmanı capture veya dominance hesaplamaz.

4.3 Data

Sorumlulukları:

- REST API ile iletişim
- Local database okuma ve yazma
- Remote ve local kaynakları koordine etme
- DTO dönüşümleri
- Batch upload yönetimi
- Retry ve senkronizasyon

Repository, feature verisinin mobil taraftaki erişim noktasıdır.

```

abstract interface class WalkRepository {
    Future<Walk> startWalk(StartWalkInput input);

    Future<void> storeLocationPoint(
        String walkId,
        LocalLocationPoint point,
    );

    Future<void> uploadPendingBatches(String walkId);

    Future<Walk> finishWalk(
        String walkId,
        DateTime endedAt,
    );
}

```

```
);  
  
Future<Walk> getWalk(String walkId);  
  
Future<WalkResult> getWalkResult(String walkId);  
}
```

4.4 Platform ve External Services

Sorumlulukları:

- Native GPS servislerine erişmek
- Background execution
- Secure storage
- Network state
- Harita SDK'sı
- Google ve Apple authentication

Bu servisler adapter arkasında kullanılmalıdır.

Uygulama kodu doğrudan plugin API'lerine dağılmamalıdır.

5. State Management

MVP'de tek bir state management yaklaşımı kullanılmalıdır.

Önerilen yaklaşım:

Riverpod

Ancak mimari Riverpod'a bağımlı tasarlanmamalıdır.

State akışı:

```
Widget  
  ↓ intent  
Controller / Notifier  
  ↓  
Repository / Service  
  ↓ result  
Immutable State  
  ↓  
Widget rebuild
```

Örnek Walk state:

```
sealed class WalkUiState {
    const WalkUiState();
}

final class WalkIdle extends WalkUiState {}

final class WalkStarting extends WalkUiState {}

final class WalkActive extends WalkUiState {
    const WalkActive({
        required this.walkId,
        required this.startedAt,
        required this.localPointCount,
        required this.pendingBatchCount,
        required this.elapsedSeconds,
    });

    final String walkId;
    final DateTime startedAt;
    final int localPointCount;
    final int pendingBatchCount;
    final int elapsedSeconds;
}

final class WalkSubmitting extends WalkUiState {}

final class WalkProcessing extends WalkUiState {
    const WalkProcessing(this.walkId);

    final String walkId;
}

final class WalkCompleted extends WalkUiState {
    const WalkCompleted(this.result);

    final WalkResult result;
}

final class WalkRejected extends WalkUiState {
    const WalkRejected(this.reasons);

    final List<String> reasons;
}

final class WalkFailure extends WalkUiState {
    const WalkFailure(this.error);
}
```

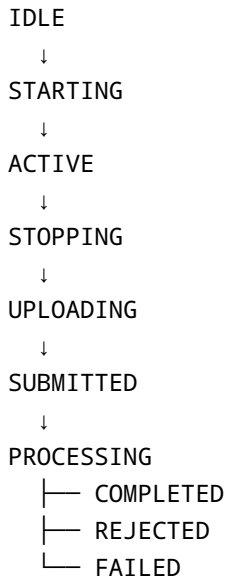
```
final AppError error;  
}
```

Kurallar:

- Widget içinde mutable global state tutulmaz.
- UI state mümkün olduğunca immutable olur.
- API DTO doğrudan UI state olmaz.
- Tek ekranın geçici state'i global provider'a taşınmaz.
- Walk aktif state'i uygulama lifecycle'ından bağımsız korunabilir olmalıdır.

6. Walk State Machine

Mobil Walk lifecycle:



Backend durumlarıyla mobil durumlar birebir olmak zorunda değildir.

Örnek eşleştirme:

Mobile	Backend
STARTING	Walk henüz oluşmadı
ACTIVE	ACTIVE
UPLOADING	ACTIVE
SUBMITTED	SUBMITTED
PROCESSING	PROCESSING

Mobile	Backend
COMPLETED	RESOLVED
REJECTED	REJECTED
FAILED	FAILED veya local hata

Walk state değişiklikleri tek bir controller tarafından yönetilmelidir.

7. Walk Başlatma Akışı

1. Kullanıcı Start Walk'a basar.
2. Aktif oturum kontrol edilir.
3. Aktif pet kontrol edilir.
4. Konum servisi açık mı kontrol edilir.
5. Gerekli izin kontrol edilir.
6. İlk güvenilir konum alınır.
7. Kullanıcının aktif şehir içinde olup olmadığı ön kontrol edilir.
8. POST /walks çağrılır.
9. Backend Walk ID döndürür.
10. Local Walk kaydı oluşturulur.
11. Location stream başlatılır.
12. UI ACTIVE durumuna geçer.

Backend Walk oluşturulmadan kalıcı GPS toplama başlatılmamalıdır.

İlk API çağrısı başarısızsa kullanıcı yürüyüşe başlamış kabul edilmez.

8. GPS Toplama Servisi

Konum servisi uygulamanın kritik platform bileşenidir.

Arayüz:

```
abstract interface class LocationService {
    Stream<DeviceLocationPoint> watch(
        LocationTrackingSettings settings,
    );

    Future<DeviceLocationPoint> getCurrent();

    Future<void> stop();
}
```

LocationTrackingSettings :

```
class LocationTrackingSettings {
    const LocationTrackingSettings({
        required this.distanceFilterMeters,
        required this.desiredAccuracy,
        required this.collectionInterval,
        required this.allowBackgroundUpdates,
    });

    final double distanceFilterMeters;
    final LocationAccuracy desiredAccuracy;
    final Duration collectionInterval;
    final bool allowBackgroundUpdates;
}
```

MVP başlangıç ayarları config üzerinden alınabilir:

Collection interval: yaklaşık 5 saniye
Distance filter: yaklaşık 5-10 metre
Accuracy: yüksek
Background updates: Walk aktifken açık

Bu değerler cihaz, pil tüketimi ve GPS kalitesi testlerinden sonra değiştirilebilir.

Native sistemin her tam zaman aralığında kesin update vermesi garanti değildir. Bu nedenle sequence ve timestamp mobil tarafından tutulur; backend gerçek segment süresini kendisi hesaplar.

9. Location Point Modeli

```
class LocalLocationPoint {
    const LocalLocationPoint({
        required this.sequence,
        required this.recordedAt,
        required this.latitude,
        required this.longitude,
        required this.accuracyMeters,
        required this.altitudeMeters,
        required this.speedMps,
        required this.isMockLocation,
        required this.syncStatus,
    });

    final int sequence;
    final DateTime recordedAt;
    final double latitude;
```

```
final double longitude;
final double accuracyMeters;
final double? altitudeMeters;
final double? speedMps;
final bool isMockLocation;
final PointSyncStatus syncStatus;
}
```

Mobil:

- Mesafeyi authoritative olarak hesaplamaz.
- Hızı güvenilir kabul etmez.
- `isMockLocation` bilgisini nihai anti-cheat kararı olarak kullanmaz.
- Noktaları toplar ve backend'e iletir.

10. Local Database

Walk sırasında toplanan GPS noktaları yalnızca memory'de tutulmamalıdır.

Uygulama:

- Çökebilir
- İşletim sistemi tarafından kapatılabilir
- Arka planda öldürülebilir
- Network bağlantısını kaybedebilir
- Kullanıcı uygulamayı yanlışlıkla kapatabilir

Bu nedenle GPS noktaları cihazda kalıcı local database'e yazılmalıdır.

Önerilen çözüm:

SQLite tabanlı local database

Flutter implementasyonu için Drift benzeri typed bir SQLite katmanı değerlendirilebilir.

MVP local entity'leri:

```
local_walks
local_walk_points
local_point_batches
sync_operations
```

11. Local Walk Entity

```
local_walks
-----
local_id
remote_walk_id
pet_id
city_id
status
started_at
ended_at
last_sequence
last_uploaded_sequence
created_at
updated_at
```

`remote_walk_id`, backend Walk oluşturulunca saklanır.

MVP'de offline olarak yeni Walk başlatılması önerilmez.

Sebep:

- Aktif şehir doğrulanmalıdır.
- Aynı oyuncunun başka aktif Walk'ı olabilir.
- Backend Walk ID gereklidir.
- Idempotency yönetimi karmaşılaşır.

Aktif Walk sırasında network kesilirse yürüyüş devam edebilir.

12. Local Point Entity

```
local_walk_points
-----
walk_local_id
sequence
recorded_at
latitude
longitude
accuracy_meters
altitude_meters
speed_mps
is_mock_location
sync_status
batch_sequence
created_at
```

Primary key:

```
walk_local_id + sequence
```

Sync durumları:

```
PENDING  
BATCHED  
UPLOADING  
UPLOADED  
FAILED
```

Raw GPS noktaları uygulama loglarına yazılmaz.

13. Offline-First Sınırı

Uygulama tam offline-first olmayacaktır.

MVP offline davranışı:

```
Walk başlatmak → internet gerekli  
Walk sırasında GPS toplamak → internet gerekli değil  
Batch upload → bağlantı gelince devam eder  
Walk bitirmek → bütün batch'ler gönderilene kadar local pending kalabilir  
Walk sonucu → internet gerekli  
Harita → son cache gösterilebilir  
Leaderboard → son cache gösterilebilir
```

Flutter'ın resmi offline-first rehberi, local ve remote veri kaynaklarının repository üzerinden koordine edilmesini ve pending yazmaların daha sonra senkronize edilmesini önerir. Bu uygulamada offline desteği yalnızca Walk sırasında veri kaybını önleyecek kapsamda tutulacaktır.

14. GPS Batch Oluşturma

Point'ler şu koşullardan biri gerçekleştiğinde batch'e dönüştürülür:

```
100 point birikti  
veya  
30 saniye geçti  
veya  
Walk bitirildi
```

Başlangıç değerleri config olmalıdır.

Batch modeli:

```
class PendingPointBatch {
    const PendingPointBatch({
        required this.walkId,
        required this.batchSequence,
        required this.firstPointSequence,
        required this.lastPointSequence,
        required this.points,
        required this.idempotencyKey,
        required this.status,
    });

    final String walkId;
    final int batchSequence;
    final int firstPointSequence;
    final int lastPointSequence;
    final List<LocalLocationPoint> points;
    final String idempotencyKey;
    final BatchSyncStatus status;
}
```

Aynı batch retry edildiğinde:

- batchSequence değişmez.
- Idempotency-Key değişmez.
- Body değişmez.

15. Batch Upload Worker

Mobil uygulama içinde hafif bir sync worker bulunacaktır.

Görevleri:

1. Pending batch'leri sırayla bulur.
2. Network uygun mu kontrol eder.
3. İlk batch'ten başlayarak gönderir.
4. Başarılı batch'i UPLOADED yapar.
5. Sonraki batch'e geçer.
6. Retry edilebilir hatada backoff uygular.
7. Kalıcı hatada Walk controller'a hata bildirir.

Aynı Walk'ın batch'leri sıralı gönderilmelidir.

Farklı Walk'lar teorik olarak paralel gönderilebilir; MVP'de tek aktif Walk olduğundan buna gerek yoktur.

Retry:

1. deneme → hemen
2. deneme → 2 saniye
3. deneme → 5 saniye
4. deneme → 15 saniye
5. deneme → 30 saniye

Sürekli network yoksa aktif retry durdurulur, bağlantı geldiğinde devam eder.

16. Walk Durdurma Akışı

1. Kullanıcı Stop Walk'a basar.
2. GPS stream durdurulur.
3. Son GPS noktaları local database'e yazılır.
4. Açık batch kapatılır.
5. Pending batch'ler backend'e gönderilir.
6. Backend'in son kabul ettiği sequence doğrulanır.
7. POST /walks/{id}/finish çağrılır.
8. Backend 202 Accepted döndürür.
9. Local Walk SUBMITTED yapılır.
10. Polling başlatılır.

Network yoksa:

GPS toplama durur.
Walk local olarak STOP_PENDING_SYNC olur.
Network geldiğinde batch'ler yüklenir.
Finish çağrısı daha sonra yapılır.

Kullanıcıya Walk'ın henüz backend'e gönderilmediği açıkça gösterilir.

17. App Lifecycle

Aktif Walk sırasında şu lifecycle durumları desteklenmelidir:

Foreground
Background

Temporarily inactive
App process recreation

Uygulama foreground'dan background'a geçtiğinde:

- Walk durmaz.
- Location tracking devam etmeye çalışır.
- Local database yazımı devam eder.
- UI timer durabilir ancak elapsed time timestamp üzerinden hesaplanır.
- Upload fırsat buldukça devam eder.

Uygulama yeniden açıldığında:

1. Local database kontrol edilir.
2. ACTIVE veya pending Walk bulunur.
3. Remote Walk state alınır.
4. GPS tracking gerekiyorsa geri kurulur.
5. Pending batch upload devam eder.
6. UI doğru state'e döner.

Memory içindeki state kaynak olarak kullanılmamalıdır.

18. Android Background Location

Android'de location permission ve background location farklı erişim seviyeleridir. Android 11 ve üzerindeki cihazlarda kullanıcı background location iznini çoğunlukla uygulama ayarları üzerinden verir; sistem izin akışı sürüme göre değişir. İzin yalnızca kullanıcı Walk özelliğini başlatırken, bağlam içinde istenmelidir.

MVP Android yaklaşımı:

Walk aktif değil:
Background location kullanılmaz.

Walk aktif:
Foreground service ile location tracking çalışır.
Kullanıcıya kalıcı sistem bildirimi gösterilir.

Android için gerekenler:

- Foreground location izni
- Precise location talebi
- Walk background'da devam edecekse gerekli background/foreground service ayarları
- Location foreground service type
- Kullanıcıya neden izin gerektiğini açıklayan ekran
- İzin reddedilirse alternatif UI

Arka planda sürekli konum toplamak batarya tüketimini artırabilir; Android de background location'ın yalnızca ana işlev için gerekli olduğunda kullanılmasını ve enerji kullanımının optimize edilmesini önerir.

19. iOS Background Location

iOS'ta background konum güncellemeleri için Location Updates background capability ve uygun Core Location yapılandırması gerekir. Apple ayrıca background kullanımının kullanıcıya açıkça anlatılmasını ve yalnızca gerekli olduğunda etkinleştirilmesini ister.

MVP iOS yaklaşımı:

```
Walk aktif değil:  
Background location kapalı.  
  
Walk aktif:  
Background location session açık.  
Walk sona erdiğinde session kapatılır.
```

İzin stratejisi:

1. Önce kullanım sırasında konum izni istenir.
2. Background gereksinimi kullanıcıya açıklanır.
3. Platformun izin akışına uygun ek izin talep edilir.
4. İzin verilmezse Walk yalnızca foreground'da güvenilir şekilde çalışabilir.

Apple mümkün olduğunda "When In Use" yetkisinin tercih edilmesini önerir; gerekli background capability etkinleştirildiğinde aktif kullanım oturumlarında background güncellemeleri desteklenebilir.

20. Permission State Model

```
enum LocationPermissionState {  
    notDetermined,  
    foregroundGranted,  
    backgroundGranted,  
    denied,  
    deniedPermanently,  
    serviceDisabled,  
    approximateOnly,  
}
```

Permission UI:

- İzin neden gerekli açıklanır.

- Sistem dialog'undan önce açıklama gösterilir.
- Permanently denied durumunda settings yönlendirmesi sunulur.
- Kullanıcı Walk başlamadan önce mevcut durum net gösterilir.
- Kullanıcı izin vermedi diye uygulamanın tümü kullanılamaz hale gelmez.
- Harita ve profil okunabilir; Walk başlatılamaz.

21. Polling

Walk finish sonrası polling controller kullanılır.

```
abstract interface class WalkPollingController {  
    Future<Walk> waitForCompletion({  
        required String walkId,  
        required Duration timeout,  
    });  
  
    void cancel();  
}
```

Polling aralığı:

```
0-10 saniye → 2 saniye  
10-30 saniye → 5 saniye  
30+ saniye → 10 saniye
```

Backend `Retry-After` header döndürürse öncelikle o değer kullanılır.

Polling şu durumlarda durur:

```
RESOLVED  
REJECTED  
FAILED  
Timeout  
Ekran kapandı ve background polling gerekmiyor  
Kullanıcı oturumu kapandı
```

Polling sonucu kaçırılırsa kullanıcı Walk geçmişinden sonucu daha sonra alabilir.

22. Territory Haritası

Harita ekranı şu katmanlardan oluşur:

Base map
Territory H3 layer
Current user marker
Optional current Walk trail
Map controls

Harita verisi:

Viewport değışti
↓
Debounce
↓
Visible bounds hesaplandı
↓
GET /cities/{cityId}/territories
↓
H3 cell response
↓
Polygon boundaries üretildi
↓
Map layer güncellendi

Mobil uygulama bütün şehir hücrelerini istemez.

23. Harita Viewport Stratejisi

Viewport sorgusu her kamera hareketinde hemen yapılmaz.

Öneri:

Camera hareketi durduktan sonra 300–500 ms debounce

Yeni sorgu yalnızca şu durumlarda yapılır:

- Viewport anlamlı şekilde değışti
- Zoom değışti
- Cache süresi doldu
- Kullanıcı refresh yaptı
- Walk tamamlandı ve territory değışti

Aynı viewport için devam eden request iptal edilebilir.

Eski request'in sonucu yeni viewport'un üzerine yazılmamalıdır.

Her request bir viewport key taşınmalıdır.

24. Territory Cache

Mobilde kısa süreli territory cache kullanılabilir.

Cache key:

```
cityId  
resolution  
viewport bucket
```

Cache'in amacı:

- Haritanın boş açılmasını önlemek
- Geri döndüğünde hızlı görüntü göstermek
- Network kullanımını azaltmak

Cache authoritative değildir.

Önerilen davranış:

```
Önce cache göster  
Arka planda backend'den yenile  
Yeni sonuç gelince haritayı güncelle
```

Sahip adı gibi profil bilgileri hücre başına gereksiz tekrar saklanmamalıdır.

25. H3 Kullanımı

Mobilde H3 kullanımı şu amaçlarla sınırlıdır:

- Backend'den gelen H3 ID'yi polygon'a çevirmek
- Harita üzerinde cell çizmek
- Viewport yardımcı hesabı
- Geçici görsel önizleme

Mobil:

- Capture hesaplamaz
- Dominance uygulamaz
- Ownership transfer yapmaz
- Route'u authoritative H3 sonucuna dönüştürmez

Backend ile mobil aynı H3 resolution bilgisine sahip olabilir; ancak backend sonucu her zaman kaynaktır.

H3 ID Dart tarafında string olarak tutulabilir.

```
typedef HexId = String;
```

Native veya Dart H3 kütüphanesine geçerken güvenli dönüşüm adapter içinde yapılır.

26. API Client

API client'in sorumlulukları:

- Base URL
- Authentication header
- Timeout
- JSON encode/decode
- Trace/request ID okuma
- Idempotency header
- Ortak hata mapping
- Retry edilebilir network hataları
- Token refresh

API client business endpoint'lerini bilmek zorunda değildir.

Feature API sınıfları kullanılır:

```
AuthApi  
WalkApi  
TerritoryApi  
LeaderboardApi  
PlayerApi  
PetApi  
CityApi
```

27. Network Retry

Her API çağrısı otomatik retry edilmez.

Retry uygulanabilecekler:

```
GET istekleri  
Idempotency-Key taşıyan güvenli POST istekleri  
Point batch upload  
Finish request
```

Retry uygulanmaması gerekenler:

- Body'nin yeniden üretilmediği istekler
- Authentication hataları
- Validation hataları
- Forbidden
- Payload too large

Retry edilebilir status:

408
429
502
503
504
Network timeout
Connection reset

Retry-After varsa dikkate alınır.

28. Authentication ve Token Storage

Access ve refresh token'lar normal local database veya shared preferences içinde açık tutulmamalıdır.

Kullanılacak katman:

SecureStorage

Platform karşılığı:

iOS → Keychain
Android → Keystore destekli secure storage

Session manager:

- Access token sağlar.
- Token expiry kontrol eder.
- Gerekirse refresh yapar.
- Aynı anda birden fazla refresh çağrısını tek işleme indirger.
- Refresh başarısızsa kullanıcı oturumunu kapatır.

Raw token loglanmaz.

29. Error Model

Mobil error sınıfları:

```
sealed class AppError {}

final class NetworkError extends AppError {}

final class UnauthorizedError extends AppError {}

final class ForbiddenError extends AppError {}

final class ValidationError extends AppError {
    ValidationError(this.fields);

    final Map<String, String> fields;
}

final class ApiBusinessError extends AppError {
    ApiBusinessError(this.code, this.details);

    final String code;
    final Map<String, dynamic>? details;
}

final class LocalStorageError extends AppError {}

final class LocationPermissionError extends AppError {}

final class UnknownError extends AppError {}
```

UI backend'in İngilizce `message` alanını doğrudan kullanıcıya göstermek zorunda değildir.

Localization hata koduna göre yapılır:

```
WALK_POINTS_INCOMPLETE
WALK_ALREADY_FINISHED
VIEWPORT_TOO_LARGE
WALK_RESULT_NOT_READY
```

Teknik hata detayları kullanıcıya gösterilmez.

30. Logging

Mobil loglarda bulunabilecekler:

```
screen
action
walkId
batchSequence
requestId
API error code
permission state
sync state
```

Loglanmayacaklar:

```
Raw latitude
Raw longitude
GPS rota listesi
Access token
Refresh token
Authorization header
Authentication credential
Tam kullanıcı adı ve hassas profil bilgileri
```

Crash reporting kullanılırsa sensitive data filtering uygulanmalıdır.

31. MVP Ekranları

MVP'de gerekli ekranlar:

1. Splash / Session Restore
2. Login
3. City Selection
4. Pet Setup
5. Main Map
6. Active Walk
7. Walk Processing
8. Walk Result
9. Walk History
10. Leaderboard
11. Profile / Stats
12. Permission Explanation

MVP dışında:

```
Guild
Chat
Events
Store
```



```
Inventory
Breed evolution
Professional walker dashboard
Push notification center
Social feed
```

32. Navigation

Önerilen ana navigation:

```
Map
Leaderboard
Profile
```

Walk aktifken navigation sınırlandırılabilir veya aktif Walk mini paneli bütün ana ekranlarda gösterilebilir.

Deep link MVP'de zorunlu değildir.

Route guard'ları:

```
Authentication
Player setup
Pet setup
City activation
Active Walk
```

33. Dependency Injection

Dependency'ler tek bootstrap noktasında oluşturulur.

```
API client
Local database
Secure storage
Location service
Repositories
Controllers
Router
```

Service locator'ın uygulama genelinde kontrolsüz kullanılmasından kaçınılır.

Riverpod kullanılırsa provider'lar dependency composition aracı olarak kullanılabilir.

Feature'lar global plugin instance'larına doğrudan erişmez.

34. Platform Adapter Sınırları

Aşağıdaki servisler interface arkasında tutulur:

```
LocationService
LocationPermissionService
SecureStorage
ConnectivityService
MapAdapter
AuthenticationProvider
AppLifecycleService
```

Bu sayede:

- Plugin değişimi sınırlı kalır.
- Test fake'leri yazılabilir.
- iOS ve Android farkları tek yerde yönetilir.
- Feature kodu platform detayını bilmez.

Her sınıf için interface oluşturulmaz.

Yalnızca platform veya external dependency sınırlarında kullanılır.

35. Test Stratejisi

Flutter resmi test rehberi; çok sayıda unit ve widget testiyle, ana kullanıcı akışlarını kapsayacak yeterli integration test kullanılmasını önerir.

35.1 Unit Tests

Öncelikli alanlar:

```
WalkController
Batch builder
Sync worker
Polling controller
Retry policy
DTO mapper
Error mapper
Session manager
Viewport debounce
```

35.2 Widget Tests

Permission state UI
Walk button durumları
Walk processing ekranı
Walk result ekranı
Network error gösterimi
Leaderboard loading/empty state

35.3 Integration Tests

Gerçek cihaz veya emülatörde:

Login
Walk start
Fake location stream
Point batch upload
Walk finish
Polling
Result gösterme
App background/foreground
Network kesilip geri gelme
Process recreation
Permission denial

35.4 Platform Tests

Background location davranışı gerçek iOS ve Android cihazlarda test edilmelidir.

Emülatör tek başına yeterli değildir.

36. Performance Kuralları

- GPS callback içinde ağır işlem yapılmaz.
- Her point'te bütün UI yeniden çizilmez.
- Point'ler batch olarak database'e yazılabilir.
- Harita polygon'ları viewport dışında render edilmez.
- Büyük H3 listeleri isolate gerektiriyorsa ölçüm sonrası ayrılır.
- Gereksiz sürekli timer kullanılmaz.
- Walk elapsed time her saniye kalıcı state'e yazılmaz.
- Map camera event'leri debounce edilir.
- API response modelleri yalnızca gerekli alanları taşır.
- Widget rebuild alanları küçük tutulur.

Performans Flutter'ın profile modunda ve gerçek cihaz üzerinde ölçülmelidir; Flutter dokümantasyonu profile modunu performans analizi için ayırır.

37. Battery Yönetimi

Battery optimizasyonları:

```
Walk aktif değilken GPS stream kapalı
Walk bitince background session kapalı
Uygun distance filter
Gereksiz yüksek frekans yok
Network batch upload
Harita camera update throttling
Background polling yok
```

GPS doğruluğu ile batarya tüketimi arasında denge load ve saha testleriyle kurulmalıdır.

İlk sürümde farklı cihaz profilleri için ayrı ayar sistemi kurulmaz.

38. Privacy

Mobil uygulama konum toplama davranışını kullanıcıya açıkça anlatmalıdır.

Temel kurallar:

- Konum yalnızca Walk sırasında toplanır.
- Walk bitince tracking durur.
- Raw GPS uygulama loguna yazılmaz.
- Kullanıcı izin vermeden konum servisi başlamaz.
- İzin nedeni onboarding sırasında açıklanır.
- Kullanıcı aktif Walk durumunu her zaman görebilir.
- Android foreground notification gizlenmez.
- Uygulama kapatılırken aktif Walk varsa uyarı gösterilir.
- Local GPS retention, backend senkronizasyonu tamamlandıktan sonra sınırlı tutulur.

39. Local Veri Temizliği

Backend tarafından Walk sonucu başarıyla alındıktan sonra:

```
Uploaded raw local points → silinebilir
Completed batches → silinebilir
Walk summary → kısa süre cache'te kalabilir
```

Öneri:

Completed raw local GPS → 24 saat sonra sil
Failed/pending GPS → senkronizasyon tamamlanana kadar tut

Logout sırasında pending Walk varsa kullanıcı uyarılmalıdır.

Pending GPS verisi doğrulanmadan silinmemelidir.

40. Recovery Senaryoları

Uygulama Walk sırasında çöktü

Local ACTIVE Walk bulunur.
Remote state alınır.
Tracking mümkünse devam ettirilir.
Kullanıcıya Walk recovery ekranı gösterilir.

Network kesildi

GPS local database'e yazılmaya devam eder.
Batch'ler PENDING kalır.
Network gelince sıralı upload devam eder.

Finish çağırısı timeout oldu

Aynı idempotency key ile tekrar gönderilir.
Backend mevcut sonucu döndürür.

Batch response kayboldu

Aynı batchSequence ve idempotency key ile yeniden gönderilir.
Duplicate point oluşmaz.

Uygulama finish sonrası kapandı

Backend worker Walk'ı işlemeye devam eder.
Uygulama açıldığında Walk state alınır.
Result hazırsa gösterilir.

Permission Walk sırasında kaldırıldı

Tracking durdurulur.
Walk kullanıcıya uyarı ile gösterilir.
Toplanan noktalar korunur.
Kullanıcı Walk'ı bitirebilir veya izni yeniden verebilir.

41. Güvenlik Sınırı

Mobil cihaz güvenilir ortam değildir.

Bu nedenle Flutter'dan gelen şu bilgiler doğrulanmadan kabul edilmez:

```
isMockLocation  
speedMps  
recordedAt  
sequence  
cityId  
duration  
distance  
location accuracy  
device state
```

Mobil yalnızca sinyal sağlar.

Backend:

- Timestamp sırasını kontrol eder.
- Mesafeyi yeniden hesaplar.
- Hızı yeniden hesaplar.
- City boundary kontrol eder.
- Duplicate noktaları engeller.
- Walk validation yapar.

42. Yapılmayacaklar

MVP'de:

Flutter içinde game engine çalıştırılmayacak.
Capture offline kesinleştirilmeyecek.
WebSocket kurulmayacak.
Push notification kurulmayacak.
Arka planda sürekli leaderboard çekilmeyecek.

Tüm şehir territory'si indirilmeyecek.
GPS yalnızca memory'de tutulmayacak.
Her GPS point ayrı HTTP isteğiyle gönderilmeyecek.
Raw GPS loglanmayacak.
Her plugin doğrudan feature kodunda kullanılmayacak.
Tam offline Walk başlangıcı desteklenmeyecek.

43. Definition of Done

Mobile Architecture uygulanmış sayılması için:

1. Feature-first klasör yapısı kullanılmalı.
2. Presentation, application ve data sorumlulukları ayrılmalı.
3. Tek state management yaklaşımı kullanılmalı.
4. Walk state machine açık olmalı.
5. GPS noktaları local database'e yazılmalı.
6. Point'ler batch halinde gönderilmeli.
7. Batch upload idempotent olmalı.
8. Network kesilince veri kaybolmamalı.
9. Finish öncesi pending batch'ler tamamlanmalı.
10. Walk sonucu polling ile alınmalı.
11. App restart sonrası aktif Walk recover edilebilmeli.
12. Background location yalnızca aktif Walk sırasında kullanılmalı.
13. Platform izin farkları adapter içinde yönetilmeli.
14. Territory viewport bazlı alınmalı.
15. H3 ID mobilde string taşınmalı.
16. Mobil authoritative game rule hesaplamamalı.
17. Token secure storage'da tutulmalı.
18. Raw GPS loglanmamalı.
19. Unit, widget ve temel integration testleri bulunmalı.
20. Gerçek cihaz background testleri yapılmalı.

44. Nihai Mimari Özeti

Pet Territory Wars Flutter MVP:

Feature-first
Katmanlı
Riverpod tabanlı state yönetimi
Repository üzerinden veri erişimi
SQLite tabanlı local GPS saklama
Batch ve idempotent upload
Aktif Walk sırasında background tracking
Asenkron backend sonucu için polling

Viewport tabanlı territory haritası
Secure token storage
Platform adapter sınırları

Ana Walk akışı:

```
graph TD; A[Start Walk] --> B[Backend Walk ID]; B --> C[GPS → Local Database]; C --> D[Point Batch Upload]; D --> E[Stop Walk]; E --> F[Pending Batch Completion]; F --> G[Finish API]; G --> H[Polling]; H --> I[Walk Result];
```

En kritik mobil prensip:

GPS noktaları önce güvenli biçimde cihazda saklanır, sonra backend'e gönderilir; oyun sonucu yalnızca backend tarafından belirlenir.